Abstract

Simulation of an Autonomous Search-and-Rescue Quadcopter

Abstract

This thesis designs and tests a simulated search-and-rescue (SAR) quadcopter in which an ESP32 acts as the drone's onboard computer and a Raspberry Pi 3 acts as the mission coordinator. No real flight takes place: motion, terrain, victims and most sensors are simulated, while the ESP32 runs real firmware and real networking, plus real IMU and barometer readings where hardware is available.

The central design choice is a three-loop architecture. A fast control loop and all safety-critical behaviour stay in deterministic code. A large language model (LLM), reached through Ollama cloud models and orchestrated with LangGraph, is limited to event-driven decisions: judging ambiguous detections, choosing the next search region and writing situation reports.

The work proceeds in nine phases, from interface definition through a deterministic baseline to the agent layer and fault-injection testing. The expected result is a working end-to-end mission demonstrator and a measured comparison between a deterministic-only coordinator and an LLM-augmented one. The comparison uses coverage, time to first detection, false-positive rate and safety-rule violations.

Keywords: search and rescue, quadcopter, ESP32, Raspberry Pi, MQTT, LangGraph, Ollama, probability map, simulation

1. Introduction

Autonomous drones can search large or dangerous areas faster than ground teams, but only if they detect people reliably, stay safe and report clearly. This thesis studies how to build such a system cheaply and test it safely in simulation.

1.1 Background

Search and rescue after disasters, or for a missing person, is time-critical. Small quadcopters carrying thermal cameras cover ground quickly and reach places people cannot. Professional SAR planning already uses two ideas that map well to software: the probability of area (POA), meaning how likely the victim is in a given cell, and the probability of detection (POD), meaning how likely a search pass would have spotted them.

Recent large language models (LLMs) can interpret context and explain their reasoning in plain language. That raises the question of whether they can help a mission coordinator without weakening flight safety.

1.2 Motivation

- **Cost and risk.** A flying prototype is expensive, hazardous and regulated. A simulation lets the software architecture be proven first.
- **Hardware limits as a design driver.** An ESP32 and a Raspberry Pi 3 are small. They force a clear split of responsibilities, as in real flight-controller and companion-computer designs.
- **Safer LLM use.** Many LLM-agent demos place the model in the control path. This work tests a structure where the model advises and deterministic code decides.

1.3 Problem statement

How can a low-cost embedded drone node and a single-board coordinator complete an autonomous SAR mission in simulation, using an LLM agent for high-level decisions, while staying safe when the cloud model is slow, unreachable or wrong?

1.4 Objectives

1. Define a fixed contract for coordinates, message topics and JSON schemas between all components.

2. Build an ESP32 drone node with a flight state machine, waypoint follower, motion model and battery model.
3. Build a simulated world with hidden victims, obstacles, decoys, wind and synthetic sensor data.
4. Build a deterministic coordinator with coverage planning, a probability map, path replanning and safety rules, and record its baseline performance.
5. Add a LangGraph agent that uses Ollama cloud models for decisions, constrained by structured outputs and a code-based safety guard.
6. Evaluate both coordinators under normal and faulty conditions using the same metrics.

1.5 Scope

The work covers software architecture, communication, planning and agent orchestration. It does not cover real flight dynamics, rotor aerodynamics, regulatory approval or field trials. Section 3 lists every simplification that follows from this scope.

2. Expected outcomes and hypotheses

The experiment should produce a working simulated SAR mission and evidence on whether an LLM agent adds value beyond a well-built deterministic planner. A result showing no benefit from the LLM is still a valid finding.

2.1 Hypotheses

- **H1, safety.** Because the safety guard is plain code, the LLM-augmented system never executes a plan that breaks geofence, altitude or battery-reserve rules.
- **H2, search efficiency.** With a reasonable prior, probability-guided search finds victims sooner than a blind lawnmower sweep.
- **H3, detection quality.** LLM-assisted verification of ambiguous detections reduces false confirmations from decoys, at the cost of added latency.
- **H4, graceful degradation.** When the cloud is unreachable, the mission still completes on the deterministic fallback with no loss of safety.

2.2 Deliverables

- ESP32 drone firmware with a flight state machine, motion model, battery model and MQTT interface.
- A world simulator that produces synthetic thermal, GPS, battery and other sensor data.
- A deterministic coordinator on the Raspberry Pi, used as the baseline.
- A LangGraph agent with structured outputs, a safety guard and human approval for high-impact actions.
- A dashboard showing the map, path, coverage, probability heatmap and detections.
- JSONL logs, replayable runs and a results comparison.

2.3 Proposed success criteria

These targets are starting points. They should be revisited once the baseline is measured in Phase 5.

Criterion	Proposed target
Executed plans breaking geofence, altitude or battery rules	0 in all runs
Missions completed and returned home with the cloud unreachable	100%
Search area covered per mission	90% or more
Victims found in standard scenarios	90% or more
False confirmations, LLM-augmented versus baseline	Equal or fewer

Criterion	Proposed target
Median replanning latency on the LLM path	Under 10 s
Link loss behaviour	Hover after about 5 s, return home after 15 to 30 s

3. Assumptions and simplifications

The experiment is deliberately simplified so that the software architecture, not the physics, is what gets tested. Every simplification below should be restated in the final write-up, because each one limits how far the results can be generalised.

3.1 Platform and physics

Assumption	What it means	Why it is acceptable
The drone does not fly	The ESP32 is the onboard computer; motion comes from a kinematic model	Flight control is a solved problem and is not the research question
Kinematic motion	The drone moves at a set speed toward waypoints, with simple acceleration limits	Avoids rotor and aerodynamic modelling
Simple wind	Wind is a constant or slowly varying velocity offset	Enough to test drift handling
Simple battery	Energy use is a function of speed and time, not of motor load	Enough to test return-to-home logic
Single drone	One drone, one coordinator	Multi-drone coordination is future work

3.2 Environment and sensing

Assumption	What it means	Why it is acceptable
Flat 2D grid world	Terrain is a grid of cells, with an optional fixed altitude for the drone	Keeps planning and the probability map tractable
Local metric frame	All positions are in metres east, north and up from a home point	Removes GPS geometry from the planner
Static obstacles and zones	Obstacles and no-fly zones do not move	Dynamic obstacle avoidance is out of scope
Simulated sensors	GPS, thermal, optical flow, battery, mmWave and CO2 are generated by the simulator	Real sensors need field conditions the project does not have
Real sensors where cheap	IMU and barometer readings may be real if wired to the ESP32	Tests real sensor timing and noise on real hardware

Assumption	What it means	Why it is acceptable
Synthetic thermal frames	Background of about 15 to 25 °C with noise, plus warm blobs for people and decoys	Enough to test detection logic, not image-processing quality
Simple noise models	Gaussian noise, dropouts and drift with hand-picked parameters	Not calibrated against real hardware

3.3 Victims and mission

- Victims are stationary by default, and a moving victim is treated as an optional fault scenario.
- The number of victims is unknown to the coordinator, but the simulator knows the ground truth.
- The search area is a known polygon with a known home point.
- The prior probability map comes from a simple rule, such as a last-known position with a spread.
- A detection is treated as a probability, not a certainty, and the simulator decides ground-truth correctness.

3.4 Communication and computing

- Communication uses MQTT over local WiFi, with injected latency and packet loss standing in for a real long-range link.
- Clocks are synchronised well enough for timestamps to be compared, using NTP or SNTP.
- The Raspberry Pi has stable power and a working internet connection unless a fault is injected.
- The world simulator may run on a laptop rather than the Pi if the Pi cannot keep up.
- Ollama cloud model names, limits and availability are treated as changeable and checked in current documentation before use.

3.5 Safety and operations

- The LLM has no authority over safety. Geofence, altitude, battery reserve and link-loss rules are enforced in code.
- A human operator is assumed to be available to approve high-impact actions, such as confirming a victim.
- Regulatory, legal and ethical requirements for real drone operations are out of scope but noted in the limitations.

4. System architecture

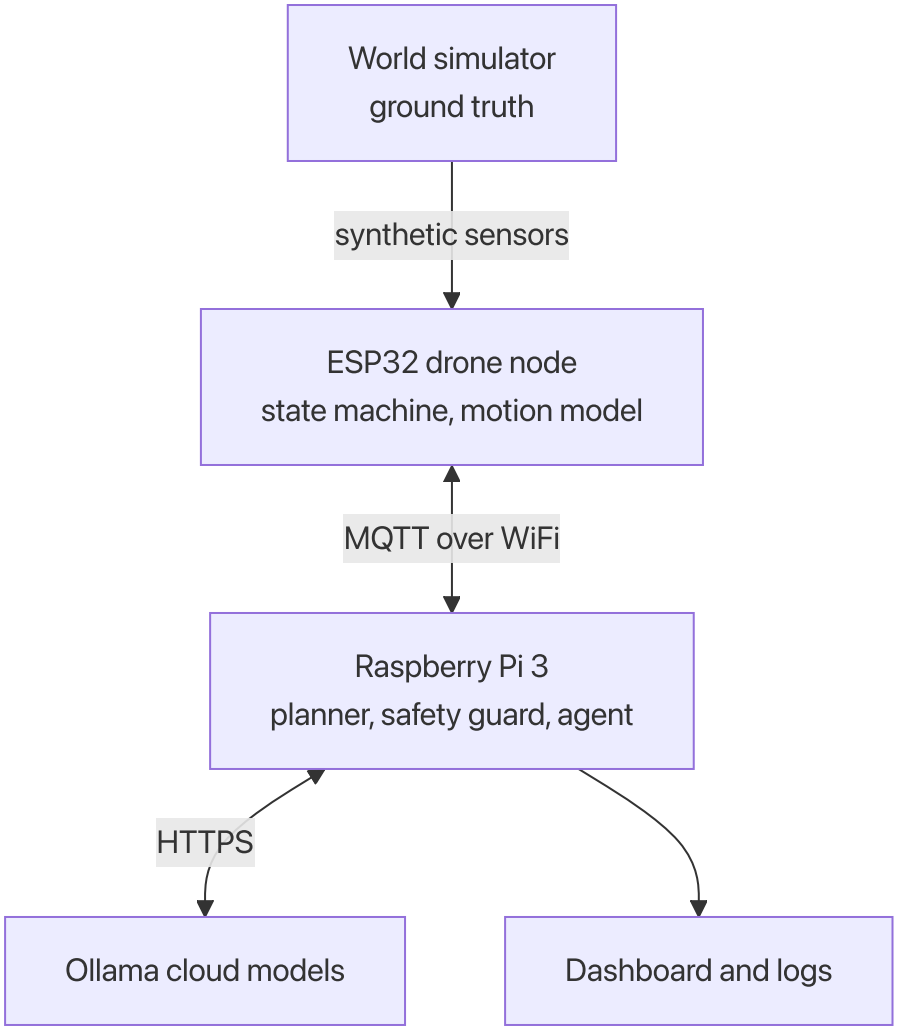
The system splits into three loops running at different speeds, and the LLM sits only in the slowest one. Cloud model calls take seconds and can fail, so they can inform decisions but must never fly the drone.

4.1 The three-loop principle

Loop	Rate	Runs on	Responsibility
Control	100 to 1000 Hz	ESP32	Attitude and position control, failsafes
Navigation and telemetry	5 to 10 Hz	ESP32 and Pi	Waypoint following, state reporting, map updates
Mission reasoning	Event-driven, seconds	Pi and Ollama cloud	Next search area, detection review, reports

In this simulation the control loop is a lightweight motion model rather than real attitude control. The loop boundary is still kept, so the design carries over to real hardware.

4.2 Component overview



The world simulator generates what the drone would sense from its current pose. The ESP32 flies the mission and reports state. The Pi decides where to search and checks every plan before it reaches the drone.

4.3 Roles of each component

Component	Main duties
ESP32 drone node	Flight state machine, waypoint follower, kinematic and battery models, sensor tasks, local failsafes, MQTT client
Raspberry Pi 3 coordinator	Mosquitto broker, map and probability grid, deterministic path planner, safety guard, LangGraph agent, dashboard, logging
World simulator	Hidden victims, decoys, obstacles, wind, synthetic thermal and other sensor frames, fault injection
Ollama cloud models	Detection review, region choice, situation reports, all through structured output

4.4 Hardware notes

The Raspberry Pi 3 has 1 GB of RAM and a slow CPU. It can host the broker, planner, dashboard and agent client, but not heavy vision models. If the world simulator becomes heavy, it moves to a laptop. An ESP32-S3 is preferable to the classic ESP32 if available, because it has more RAM and camera support.

4.5 Design rules

- The LLM outputs structured intent only. Deterministic code converts intent into waypoints.
- Every plan passes a code-based safety guard that can veto it.
- The agent runs on events such as a detection, a finished region, low battery or a link event, not on a timer.
- If the cloud is unreachable, the deterministic sweep continues.
- The Ollama API key lives only on the Pi.

5. SAR quadcopter requirements

A SAR quadcopter needs seven capabilities: flight, navigation, perception, communication, power management, safety and mission logic. The simulation implements each one either for real or as a model.

5.1 Required subsystems

Subsystem	Real-world need	In this simulation
Flight core	IMU, barometer, magnetometer, motors, attitude and position control	Kinematic model, optional real IMU and barometer
Navigation	GPS outdoors; optical flow and rangefinder where GPS is denied; a state estimator	Simulated GPS with noise and dropouts; complementary or simple Kalman filter
Perception	Thermal camera as primary human detector, RGB for confirmation, optional radar, CO2 and audio	Synthetic thermal frames; sample images from public datasets for RGB
Communication	Telemetry link, command link, heartbeats	MQTT over WiFi with fault injection
Power	Battery monitoring, endurance estimate, return-to-home reserve	Battery model based on speed and time
Safety	Geofence, altitude limit, obstacle avoidance, failsafes	Code-based guard on the Pi plus state machine on the ESP32
Mission logic	Area coverage, detection handling, target marking, reporting	Coordinator and LangGraph agent

5.2 Sensor suite and parameters

Sensor	Purpose	Parameters and rate	Real or simulated
IMU (MPU6050, ICM-20948, BNO055)	Attitude	Accel in m/s², gyro in rad/s, 200 to 1000 Hz	Real if wired
Barometer (BMP280, BMP388)	Altitude	Pressure in Pa, temperature, 25 to 50 Hz	Real if wired
Magnetometer	Heading	Field in µT, 50 Hz	Real, noisy indoors

Sensor	Purpose	Parameters and rate	Real or simulated
GPS (u-blox NEO-M8N class)	Position	Lat, lon, altitude, fix type, satellites, HDOP, 5 to 10 Hz	Simulated
Optical flow (PMW3901) and ToF (VL53L1X)	Velocity and height without GPS	Flow in px/frame, range in mm, 30 to 100 Hz	Simulated
Battery monitor (INA219, INA226)	Endurance	Voltage, current, mAh used, 10 Hz	Simulated
Thermal camera (MLX90640, Lepton)	Detect humans	Per-pixel °C, field of view, 1 to 8 Hz	Synthetic frames
RGB camera (OV2640, Pi camera)	Visual confirmation	JPEG frames, 1 to 5 Hz	Dataset images
mmWave radar (MR60BHA1, LD2410)	Life signs through debris	Presence, breathing rate, range 1 to 6 m	Simulated
CO2 sensor (SCD40)	Trapped-victim hint	ppm, 1 Hz	Simulated
Obstacle ToF or ultrasonic	Avoidance	Range in m, 10 to 30 Hz	Simulated
WiFi or BLE RSSI from a phone or beacon	Coarse localisation	dBm, 1 to 5 Hz	Simulated

5.3 Synthetic data plan

- **Thermal.** Generate frames with a 15 to 25 °C background and sensor noise, then add Gaussian blobs at 30 to 36 °C for people. Add decoys such as sunlit rocks and animals to test false positives.
- **RGB.** Aerial person-detection datasets such as HERIDAL, SARD and VisDrone exist, and HIT-UAV provides thermal images. Their current terms and availability should be checked before use.
- **GPS.** Generate ground truth in the local frame, then add noise, dropouts and slow drift.

5.4 Minimum viable sensor set

If time is short, the project remains valid with four sensors: simulated GPS, simulated battery, synthetic thermal frames and a real or simulated IMU. The rest can be added as extensions without changing the architecture.

6. Methodology

The methods below are chosen to be simple, testable and replaceable. Each has a deterministic core so results can be reproduced and the LLM's contribution can be isolated.

6.1 Coordinates and position estimation

All planning uses a local east-north-up (ENU) frame in metres, measured from a home point. Latitude and longitude are computed only for display. This keeps GPS geometry out of the planner.

The ESP32 fuses IMU, barometer and GPS with a complementary filter, or a simple per-axis Kalman filter. When simulated GPS drops out, it dead-reckons from optical flow and lets position uncertainty grow. Reporting that uncertainty to the coordinator is a deliberate

demo feature.

6.2 Coverage planning

The baseline search is a lawnmower (boustrophedon) sweep over the search polygon. The swath width depends on flight height h and camera field of view FOV:

$$W = 2h \tan\left(\frac{\mathrm{FOV}}{2}\right)$$

For a 55° field of view at 30 m, the swath is about 31 m. Adjacent passes overlap by 20 to 30% to allow for position error and detection misses.

6.3 Probability map

The area is divided into grid cells, each holding the probability that a victim is there. Priors come from a last known position, terrain or a subject profile. After each pass, cells that were covered without a detection are down-weighted by the detection probability p_d of that pass, then all cells are renormalised:

$$P'(i) = \frac{P(i),\big(1 - p_d(i)\big)}{\sum_j P(j),\big(1 - p_d(j)\big)}$$

Here p_d is zero for cells the drone did not cover. The planner then favours high-probability cells over blind sweeping, and the LLM gets a meaningful summary to reason about.

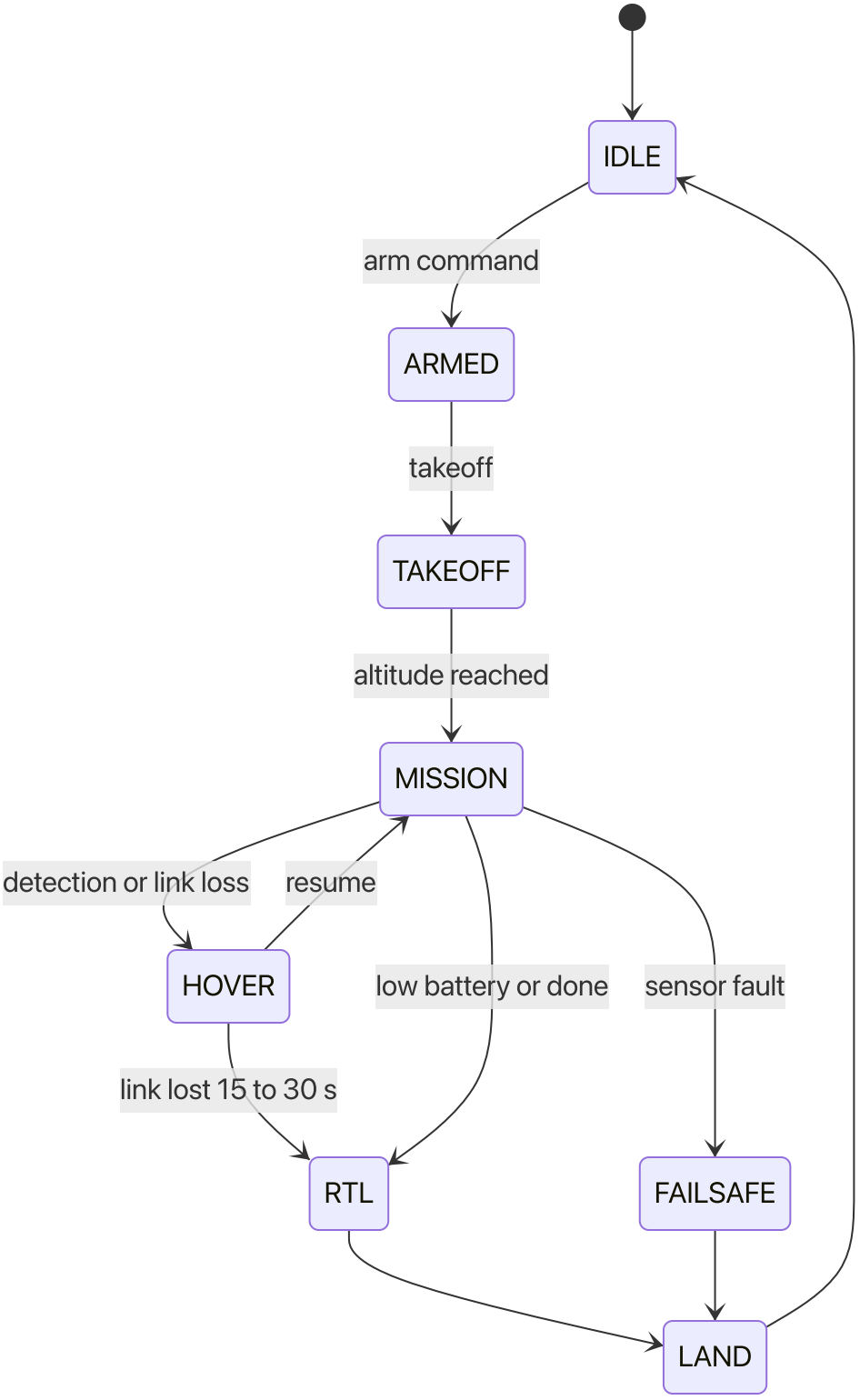
6.4 Detection handling

When the thermal frames show a candidate, the drone breaks off the sweep, hovers or orbits for a confirming pass, then reports the detection with a confidence value. The coordinator scores it, optionally asks the LLM to review it in context, and asks the human operator to approve marking a confirmed victim. The drone then resumes the mission.

6.5 Obstacles, geofence and battery

- A *or D* Lite on the grid routes around obstacles and no-fly zones.
- Every waypoint is checked against the geofence and altitude limits before it is sent.
- Return-to-home starts when the remaining energy falls below the energy needed to fly home plus a reserve of 25 to 30%.

6.6 Flight state machine



Each transition has a defined trigger, so the fault scenarios in Section 9 can test them one by one.

6.7 Communication

The link is MQTT over WiFi, with Mosquitto on the Pi and the ESP32 as a client. It is simple, easy to debug and easy to inject faults into. The messages could later be wrapped in MAVLink for realism.

Topic	Rate	QoS	Purpose
sar/d1/telemetry/state	10 Hz	0	Position, velocity, mode, battery
sar/d1/sensors/thermal	2 Hz	0	Thermal frames or summaries

Topic	Rate	QoS	Purpose
sar/d1/events/detection	On demand	1	Candidate victim reports
sar/d1/heartbeat	1 Hz	0	Link health
sar/d1/cmd/waypoint	On demand	1	Waypoint commands
sar/d1/cmd/mode	On demand	1	MISSION, HOVER, RTL, LAND

Example state message:

```
{"t": 1727100000.12, "seq": 4211, "mode": "MISSION",
 "pos": [12.4, -8.1, 30.0], "vel": [1.9, 0.0, 0.0], "yaw": 90.0,
 "gps": {"fix": 3, "sats": 9, "hdop": 1.1},
 "batt": {"v": 11.4, "pct": 62}, "wp_idx": 5, "link_rssi": -58}
```

If heartbeats stop for about 5 seconds, the drone hovers. After 15 to 30 seconds, it flies home. The Pi injects latency and packet loss to test this. Real WiFi reaches roughly 100 m, and real SAR drones would use LoRa, 900 MHz radios or cellular, so the write-up should say so. Every message carries a timestamp, and MQTT authentication is enabled.

6.8 Logging and replay

All messages, decisions and agent inputs and outputs are written to JSONL files. Any run can then be replayed and compared with another under identical conditions.

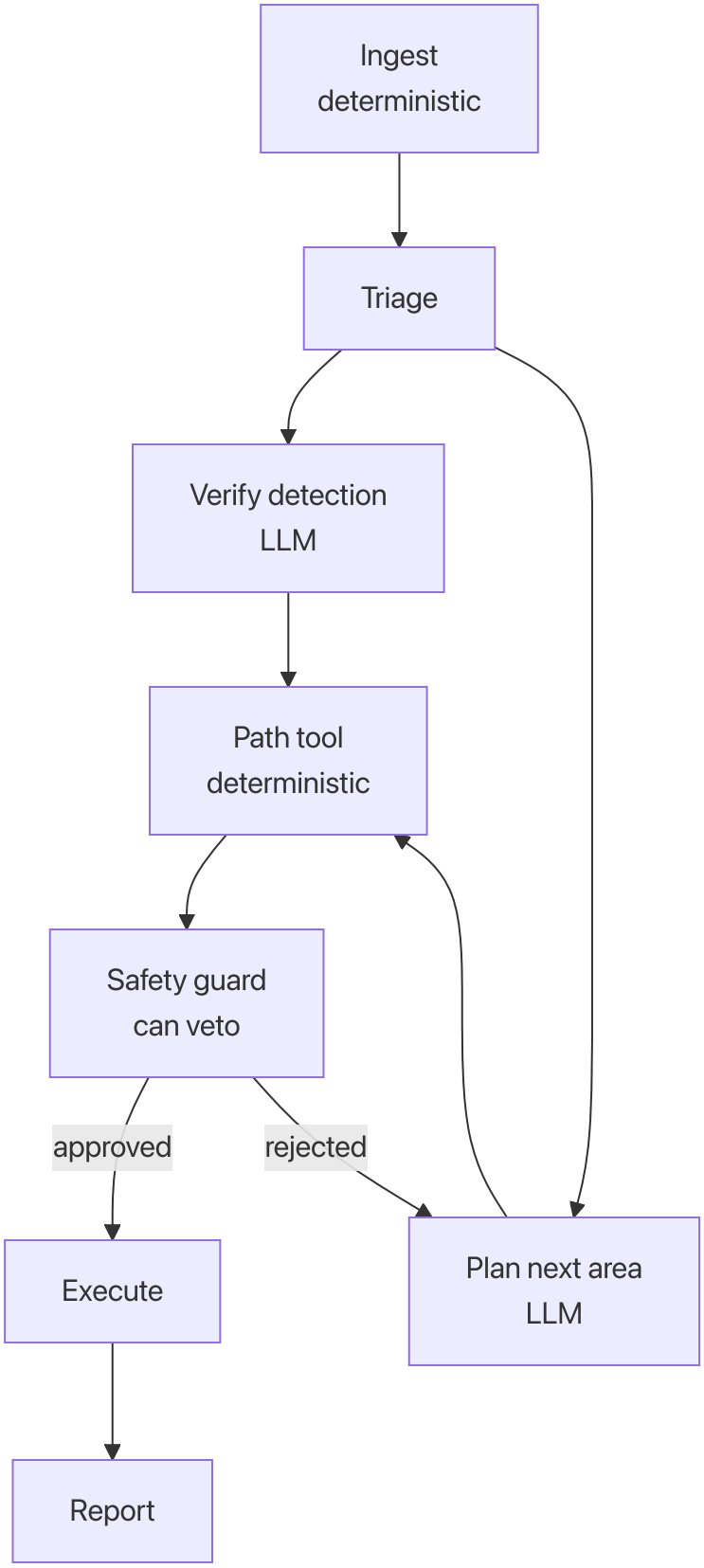
7. Agent design

The agent is a LangGraph workflow on the Pi that calls Ollama cloud models for judgement, and hands everything else to deterministic tools. Its output is always a structured intent, never raw flight commands.

7.1 Where the LLM helps and where it must not

The LLM may	The LLM must not
Interpret a detection in context, such as a low-confidence blob next to a sunlit rock	Compute waypoints
Choose which region to search next from the probability map, battery and time	Perform obstacle avoidance
Explain its decisions and write situation reports	Override safety checks such as battery, geofence or altitude
Choose among predefined tools when something unexpected happens	Handle anything time-critical

7.2 Graph structure



A rejected plan returns to the planning node with the guard's reason attached, so the model can propose an alternative. After a small number of rejections the coordinator falls back to the deterministic sweep.

7.3 Structured intent

The model returns a schema-validated object. Deterministic code turns it into waypoints.

```
class Intent(BaseModel):
    action: Literal["search_region", "confirm_detection", "return_home", "hold"]
```

```
    region_id: int | None = None
    rationale: str
```

7.4 Reliability rules

- **Code-based safety guard.** Battery, geofence and altitude limits are never up to the model.
- **Event triggers.** The agent runs on a detection, a completed region, low battery or a link event, which respects latency and rate limits.
- **Checkpointing.** LangGraph's checkpointer, backed by SQLite, stores state so a run can be resumed or replayed.
- **Human approval.** A LangGraph interrupt pauses the graph before high-impact actions, such as marking a confirmed victim.
- **Cloud fallback.** If the cloud call fails or times out, the deterministic sweep continues. This also provides the comparison baseline.

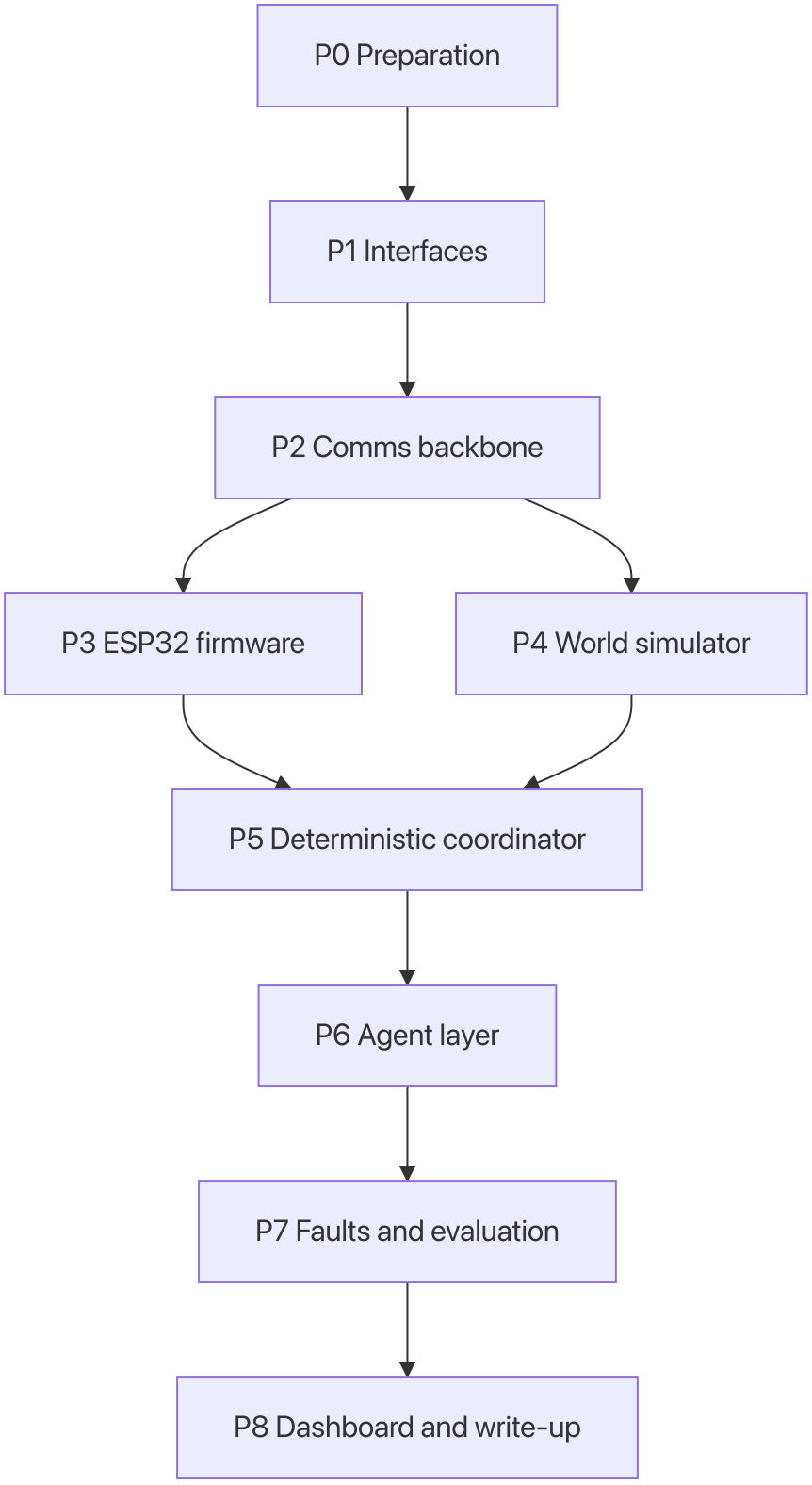
7.5 Model access

There are two ways to reach Ollama cloud models. The Pi can call the hosted Ollama API directly with an API key, through the `ChatOllama` class in `langchain-ollama`. Alternatively, the Pi's local Ollama can be signed in and can serve a cloud-tagged model. Model names, tool-calling support and rate limits change, so the current Ollama documentation should be checked before Phase 6. A vision-capable model is needed only if the agent is to inspect RGB frames.

8. Phase-wise implementation plan

The project runs in nine phases, numbered 0 to 8, and the LLM is not touched until Phase 6. Each phase ends with an exit criterion that must pass before the next begins, except that Phases 3 and 4 can run in parallel.

8.1 Overview



The durations below are rough estimates for part-time work and should be adjusted to real availability. With Phases 3 and 4 overlapping, the total is about 16 weeks.

Phase	Indicative duration	Main output	Exit gate
0. Preparation	1 week	Working toolchains on both boards	Hello-world on ESP32 and Pi; Pi reaches Ollama
1. Interfaces	1 week	Frame, topics and JSON schemas	Sample messages validate on both sides

Phase	Indicative duration	Main output	Exit gate
2. Comms backbone	1 week	MQTT link with logging and fault injection	30-minute run with no manual restart
3. ESP32 firmware	3 weeks	Drone node with state machine and models	Follows scripted waypoints; failsafes trigger
4. World simulator	2 weeks, parallel with 3	Hidden victims, decoys, synthetic sensors	Seeded scenarios reproduce identically
5. Deterministic coordinator	3 weeks	Full mission with no LLM	Baseline metrics recorded on five or more scenarios
6. Agent layer	3 weeks	LangGraph agent with guard and approval	Same scenarios completed; unsafe intents vetoed
7. Faults and evaluation	2 weeks	Comparison of both coordinators	Results table and plots complete
8. Dashboard and write-up	2 weeks	Dashboard, demo run, final thesis text	Recorded demo and finished document

8.2 Phase 0: Preparation and hardware bring-up

Objective. Remove setup risk before any design work.

- Set up the ESP32 toolchain, using PlatformIO with Arduino or ESP-IDF, and flash a test program.
- Set up the Pi with its operating system, a Python virtual environment and Mosquitto.
- Create a version-controlled repository with separate folders for firmware, simulator, coordinator and docs.
- If an IMU or barometer is available, read it on the ESP32 and print values.
- Make one test call to an Ollama cloud model from the Pi, as an early check that access works.

Deliverables. Repository, working toolchains, a note on the chosen model and access method.

Exit criterion. Both boards run test code, and the Pi receives a reply from an Ollama cloud model.

8.3 Phase 1: Interface definition

Objective. Fix the contract so both sides can be built independently.

- Define the ENU coordinate frame, units and the home point.
- Write JSON schemas for state, detection, waypoint, mode command and heartbeat messages, each with a version field.
- Finalise the topic map from Section 6.7 and the flight state machine from Section 6.6.
- Create a configuration file for world size, grid resolution, speeds and battery parameters.

Deliverables. An interface specification, schema files and sample messages.

Exit criterion. Sample messages validate against the schemas in both Python and the ESP32 code.

8.4 Phase 2: Communication backbone

Objective. Prove the link is reliable before building on it.

- Configure Mosquitto with authentication.
- Have the ESP32 publish fake telemetry and heartbeats, and have the Pi subscribe and log to JSONL.
- Add reconnect logic on both sides and test it by restarting the broker and the WiFi.
- Build a small latency and packet-loss injector on the Pi.
- Measure round-trip time and message loss under normal and degraded conditions.

Deliverables. Working link, JSONL logger, fault injector and a baseline measurement of latency.

Exit criterion. A 30-minute run completes with no manual restarts, and the link recovers after a deliberate outage.

8.5 Phase 3: ESP32 drone firmware

Objective. Make the ESP32 behave like a drone, without real flight.

- Build FreeRTOS tasks for sensors, control and communication.
- Implement the state machine, the waypoint follower, a kinematic motion model and a battery model.
- Implement heartbeat-loss handling: hover after about 5 s, return home after 15 to 30 s.
- Add real IMU and barometer readings with a complementary filter, if the hardware is available.
- Report state at 10 Hz in the agreed schema.

Deliverables. Firmware with unit-tested state transitions and a script that sends test waypoints.

Exit criterion. The drone follows a scripted waypoint list, transitions correctly on every trigger in the state machine, and hovers and returns home on simulated link loss.

8.6 Phase 4: World simulator

Objective. Create a repeatable world that generates what the drone would sense.

- Build a grid map with hidden victims, decoys such as warm rocks and animals, obstacles, no-fly zones and wind.
- Generate synthetic thermal frames from the drone's pose, with noise and blobs at 30 to 36 °C for people.
- Generate noisy GPS with dropouts and drift, and a battery model that matches the ESP32's.
- Add hooks to inject faults on command, and log ground truth for every run.
- Make every scenario reproducible from a random seed.

Deliverables. Simulator, scenario files and a ground-truth log format.

Exit criterion. For a given pose, the simulator returns a thermal frame in which a victim within the field of view is visible, and the same seed reproduces identical results.

8.7 Phase 5: Deterministic coordinator

Objective. Achieve a complete autonomous mission with no LLM, and measure it.

- Implement the lawnmower planner with the swath and overlap rules from Section 6.2.
- Implement the probability map and its update rule from Section 6.3.
- Implement A* replanning around obstacles and no-fly zones.
- Implement the geofence, altitude limits and battery-aware return-to-home.
- Implement detection handling: confirming pass, marking and resuming.
- Build the metrics collector for the measures in Section 9.

Deliverables. A mission runner and baseline results on at least five seeded scenarios.

Exit criterion. The mission runs from takeoff to landing without intervention and finds victims in the standard scenarios. This is the gate for Phase 6: the agent work does not begin until this passes, because a failure in the plumbing would otherwise be blamed on the model.

8.8 Phase 6: Agent layer

Objective. Add LLM judgement without adding LLM risk.

- Wrap the planner, probability map and safety checks as tools with typed inputs and outputs.
- Define the Pydantic schemas for structured intent.
- Build the LangGraph graph from Section 7.2, with a plain-code safety guard that can veto any plan.
- Add an interrupt for human approval before marking a confirmed victim.
- Add the SQLite checkpointer and the cloud-failure fallback.
- Develop and tune prompts offline by replaying recorded events from Phase 5, then move to live runs.
- Keep prompts and model names under version control so runs can be compared.

Deliverables. Agent code, prompt files, unit tests for the guard and a replay harness.

Exit criterion. The agent completes the same scenarios as the baseline, the guard vetoes deliberately unsafe intents in tests, and the mission still finishes with the internet blocked.

8.9 Phase 7: Faults and evaluation

Objective. Compare the two coordinators fairly and stress the system.

- Run each fault scenario from Section 9 for both coordinators, with the same seeds and several repeats per scenario.
- Record all metrics and compute averages and spread.
- Note cases where the agent helped, where it hurt, and where it made no difference.

Deliverables. Results tables, plots and a short analysis of each hypothesis from Section 2.1.

Exit criterion. Every hypothesis has a supporting or refuting result, and any unsafe behaviour is explained.

8.10 Phase 8: Dashboard, demonstration and write-up

Objective. Make the work visible and finish the thesis.

- Build a Streamlit or Flask dashboard with a 2D or Leaflet map showing the drone, path, coverage, probability heatmap and detections.
- Add a panel with the agent's latest reasoning and any pending approval.
- Record a demonstration run and prepare a reproducible demo script.
- Complete the write-up, including limitations and the simplifications from Section 3.

Deliverables. Dashboard, recorded demo, final document and repository.

Exit criterion. A new viewer can run the demo from the repository instructions and understand what the system is doing.

8.11 Working rule

Most projects like this fail by starting with the agent instead of the plumbing. If a phase is running late, shrink its scope, not its exit criterion.

9. Evaluation plan

The evaluation compares two coordinators on identical scenarios and seeds: a deterministic-only baseline, and the LLM-augmented agent. Using the same seeds means any difference comes from the coordinator, not from luck in the world.

9.1 Scenarios and fault injection

Scenario	What is varied	What it tests
Nominal	One victim, no faults	Baseline coverage and detection

Scenario	What is varied	What it tests
Multiple victims	Two to four victims	Detection handling and re-planning
Decoys	Warm rocks and animals	False-positive handling (H3)
Biased prior	Prior map that is partly wrong	Value of probability-guided search (H2)
GPS dropout	Loss of GPS for a set interval	Dead reckoning and uncertainty reporting
Sensor noise spike	Short burst of thermal or IMU noise	Robustness of detection
Packet loss and latency	Injected on the link	Link handling
Link loss	Heartbeats stop	Hover and return-home behaviour (H1, H4)
Low battery mid-mission	Reduced starting charge	Return-to-home reserve (H1)
Wind	Constant and gusting offsets	Path following and energy use
Cloud outage	Internet blocked	Fallback path (H4)
Moving victim	Slow random walk	Limits of the static assumption

9.2 Metrics

Metric	Meaning
Coverage percentage	Share of the search area seen by the sensor
Time to first detection	Seconds from takeoff to the first correct detection
Victims found	Correct confirmations out of the true count
False positive rate	Wrong confirmations per mission
Battery remaining at finish	Percentage left on landing
Replanning latency	Time from event to a new approved plan
Safety violations	Executed plans breaking geofence, altitude or battery rules
Mission completion	Whether the drone landed safely after finishing or aborting

9.3 Method

- Run every scenario for both coordinators with the same set of seeds, using several repeats per scenario.
- Report averages together with spread, not averages alone.
- Log all agent inputs and outputs so unexpected decisions can be reviewed after the fact.

- Keep the model, prompt version and settings fixed within a comparison, and record them with the results.

9.4 Threats to validity

Results from a simplified simulator may not carry over to real flights. Cloud model behaviour can change between runs and over time, which affects repeatability. The synthetic thermal data is easier than real imagery. These points should be stated alongside every result.

10. Risks and limitations

The biggest project risks are hardware limits on the Pi, dependence on a cloud service, and scope creep. Each has a planned mitigation.

10.1 Project risks

Risk	Effect	Mitigation
Raspberry Pi 3 too slow or short of RAM	Laggy planner, dashboard or simulator	Move the simulator to a laptop; keep the dashboard light
Cloud latency, outage or rate limits	Slow or missing agent decisions	Event-driven triggers, timeouts and the deterministic fallback
Model names or behaviour change	Results differ between runs	Pin and record model names and prompt versions; check current Ollama documentation
LLM returns invalid or unsafe intents	Bad plans	Schema validation, code-based guard, retry limit, then fallback
Unstable WiFi	Dropped messages and false link-loss events	Reconnect logic, tuned heartbeat thresholds, a stable test network
ESP32 timing jitter	Uneven state updates	Separate tasks, fixed-rate scheduling, modest message rates
Scope creep from extra sensors or features	Missed schedule	Minimum viable sensor set and phase exit gates
Synthetic data too easy	Overstated results	Add decoys and noise; optionally test with dataset images

10.2 Limitations

- The drone never flies, so results say nothing about real aerodynamics, vibration or wind response.
- One drone and one simulated environment cannot represent the range of real search terrain.
- Simplified sensor and battery models are not calibrated against real hardware.
- WiFi range and reliability are far below what a real SAR link needs.
- Real drone operations involve regulation, airspace rules and ethical questions about privacy and responsibility, none of which this project addresses.
- Real rescue decisions must stay with human operators. The human-approval step is a demonstration of that principle, not a substitute for it.

11. Conclusion and future work

This project builds a low-cost, simulated SAR quadcopter and uses it to test one question: where does an LLM add value to an autonomous search mission, and where must deterministic code stay in charge? The proposed answer is a three-loop design in which the model advises on detections and search priorities while plain code plans paths and enforces safety.

The phased plan keeps risk low. It builds and measures a deterministic baseline first, and only then adds the agent, so any change in performance can be traced to the model. Whatever the results show, the project yields a reusable architecture, a replayable test bench and a fair comparison.

11.1 Future work

- Replace the kinematic model with a real flight controller stack such as PX4 or ArduPilot, connected through MAVLink.
- Use a real thermal camera, and run a small person-detection model on the ESP32-S3 or a stronger companion computer.
- Extend to several drones with shared probability maps and task allocation.
- Use a long-range link such as LoRa, a 900 MHz radio or cellular, and study its effect on agent behaviour.
- Add moving victims, three-dimensional terrain and dynamic obstacles.
- Compare cloud models against smaller local models for latency, cost and decision quality.
- Move from software-in-the-loop to hardware-in-the-loop tests, and then to controlled field trials within applicable regulations.

References and resources

These resources are listed by name from prior knowledge and have not been opened or checked for this draft. Before citing any of them formally, confirm the current link, version and licence, and replace this list with proper citations.

Category	Resource	Use in this project
Datasets	HERIDAL, SARD, VisDrone	Aerial person images for RGB confirmation tests
Datasets	HIT-UAV	Thermal drone imagery for realism checks on synthetic frames
Messaging	MQTT and the Mosquitto broker	Drone-to-coordinator link
Messaging	MAVLink	Optional message format for realism and future work
Agents	LangGraph	Workflow orchestration, checkpointing and interrupts
Agents	Ollama documentation	Current cloud model names, tool-calling support and limits
Agents	langchain-ollama (`ChatOllama`)	Client for calling Ollama models from LangGraph
Firmware	ESP-IDF, Arduino-ESP32, PlatformIO, FreeRTOS	ESP32 development and task scheduling
Flight software	PX4, ArduPilot	Future replacement of the kinematic model
Planning	Lawnmower coverage, A, *D* Lite, POA and POD search theory	Coverage, routing and probability-map methods

Category	Resource	Use in this project
Dashboard	Streamlit, Flask, Leaflet	Mission visualisation